What's new in 2.0.0 (April 3, 2023)

These are the changes in pandas 2.0.0. See Release notes for a full changelog including other versions of pandas.

Enhancements

Installing optional dependencies with pip extras

When installing pandas using pip, sets of optional dependencies can also be installed by specifying extras.

```
pip install "pandas[performance, aws]>=2.0.0"
```

The available extras, found in the installation guide, are `[all, performance, computation, fss, aws, gcp, excel, parquet, feather, hdf5, spss, postgresql, mysql, sql-other, html, xml, plot, output_formatting, clipboard, compression, test]` (GH39164).

`Index` can now hold numpy numeric dtypes

It is now possible to use any numpy numeric dtype in a `Index` (GH42717).

Previously it was only possible to use `int64`, `uint64` & `float64` dtypes:

```
In [1]: pd.Index([1, 2, 3], dtype=np.int8)
Out[1]: Int64Index([1, 2, 3], dtype="int64")
In [2]: pd.Index([1, 2, 3], dtype=np.uint16)
Out[2]: UInt64Index([1, 2, 3], dtype="uint64")
In [3]: pd.Index([1, 2, 3], dtype=np.float32)
Out[3]: Float64Index([1.0, 2.0, 3.0], dtype="float64")
```

`Int64Index`, `UInt64Index` & `Float64Index` were deprecated in pandas version 1.4 and have now been removed. Instead `Index` should be used directly, and can it now take all numpy numeric dtypes, i.e. `int8` / `int16` / `int32` / `int64` / `uint8` / `uint16` / `uint32` / `uint64` / `float32` / `float64` dtypes:

```
In [1]: pd.Index([1, 2, 3], dtype=np.int8)
Out[1]: Index([1, 2, 3], dtype='int8')

In [2]: pd.Index([1, 2, 3], dtype=np.uint16)
Out[2]: Index([1, 2, 3], dtype='uint16')

In [3]: pd.Index([1, 2, 3], dtype=np.float32)
Out[3]: Index([1.0, 2.0, 3.0], dtype='float32')
```

The ability for `Index` to hold the numpy numeric dtypes has meant some changes in Pandas functionality. In particular, operations that previously were forced to create 64-bit indexes, can now create indexes with lower bit sizes, e.g. 32-bit indexes.

Below is a possibly non-exhaustive list of changes:

1. Instantiating using a numpy numeric array now follows the dtype of the numpy array. Previously, all indexes created from numpy numeric arrays were forced to 64-bit. Now, for example, `Index(np.array([1, 2, 3]))` will be `int32` on 32-bit systems, where it previously would have been `int64` even on 32-bit systems. Instantiating `Index` using a list of numbers will still return 64bit dtypes, e.g. `Index([1, 2, 3])` will have a `int64` dtype, which is the same as previously.
2. The various numeric datetime attributes of `DatetimeIndex` (`day`, `month`, `year` etc.) were previously in of dtype `int64`, while they were `int32` for `arrays.DatetimeArray`. They are now `int32` on `DatetimeIndex` also:

- Removed deprecated `Index.asi8()` (GH37877)
- Enforced deprecation changing behavior when passing `datetime64[ns]` dtype data and timezone-aware dtype to `Series`, interpreting the values as wall-times instead of UTC times, matching `DatetimeIndex` behavior (GH41662)
- Enforced deprecation changing behavior when applying a numpy ufunc on multiple non-aligned (on the index or columns) `DataFrame` that will now align the inputs first (GH39239)
- Removed deprecated `DataFrame._AXIS_NUMBERS()`, `DataFrame._AXIS_NAMES()`, `Series._AXIS_NUMBERS()`, `Series._AXIS_NAMES()` (GH33637)
- Removed deprecated `Index.to_native_types()`, use `obj.astype(str)` instead (GH36418)
- Removed deprecated `Series.iteritems()`, `DataFrame.iteritems()`, use `obj.items` instead (GH45321)
- Removed deprecated `DataFrame.lookup()` (GH35224)
- Removed deprecated `Series.append()`, `DataFrame.append()`, use `concat()` instead (GH35407)
- Removed deprecated `Series.iteritems()`, `DataFrame.iteritems()` and `HDFStore.iteritems()` use `obj.items` instead (GH45321)
- Removed deprecated `DatetimeIndex.union_many()` (GH45018)
- Removed deprecated `weekofyear` and `week` attributes of `DatetimeArray`, `DatetimeIndex` and `dt` accessor in favor of `isocalendar().week` (GH33595)
- Removed deprecated `RangeIndex._start()`, `RangeIndex._stop()`, `RangeIndex._step()`, use `start`, `stop`, `step` instead (GH30482)
- Removed deprecated `DatetimeIndex.to_perioddelta()`, Use `dtindex - dtindex.to_period(freq).to_timestamp()` instead (GH34853)
- Removed deprecated `Styler.hide_index()` and `Styler.hide_columns()` (GH49397)
- Removed deprecated `Styler.set_na_rep()` and `Styler.set_precision()` (GH49397)
- Removed deprecated `Styler.where()` (GH49397)
- Removed deprecated `Styler.render()` (GH49397)
- Removed deprecated argument `col_space` in `DataFrame.to_latex()` (GH47970)
- Removed deprecated argument `null_color` in `Styler.highlight_null()` (GH49397)
- Removed deprecated argument `check_less_precise` in `testing.assert_frame_equal()`, `testing.assert_extension_array_equal()`, `testing.assert_series_equal()`, `testing.assert_index_equal()` (GH30562)
- Removed deprecated `null_counts` argument in `DataFrame.info()`. Use `show_counts` instead (GH37999)
- Removed deprecated `Index.is_monotonic()`, and `Series.is_monotonic()`; use `obj.is_monotonic_increasing` instead (GH45422)
- Removed deprecated `Index.is_all_dates()` (GH36697)
- Enforced deprecation disallowing passing a timezone-aware `Timestamp` and `dtype="datetime64[ns]"` to `Series` or `DataFrame` constructors (GH41555)
- Enforced deprecation disallowing passing a sequence of timezone-aware values and `dtype="datetime64[ns]"` to to `Series` or `DataFrame` constructors (GH41555)
- Enforced deprecation disallowing `numpy.ma.mrecords.MaskedRecords` in the `DataFrame` constructor; pass `"{name: data[name] for name in data.dtype.names}` instead (GH40363)
- Enforced deprecation disallowing unit-less "datetime64" dtype in `Series.astype()` and `DataFrame.astype()` (GH47844)
- Enforced deprecation disallowing using `.astype` to convert a `datetime64[ns]` `Series`, `DataFrame`, or `DatetimeIndex` to timezone-aware dtype, use `obj.tz_localize` or `ser.dt.tz_localize` instead (GH39258)
- Enforced deprecation disallowing using `.astype` to convert a timezone-aware `Series`, `DataFrame`, or `DatetimeIndex` to timezone-naive `datetime64[ns]` dtype, use `obj.tz_localize(None)` or `obj.tz_convert("UTC").tz_localize(None)` instead (GH39258)
- Enforced deprecation disallowing passing non boolean argument to sort in `concat()` (GH44629)
- Removed Date parser functions `parse_date_time()`, `parse_date_fields()`, `parse_all_fields()` and `generic_parser()` (GH24518)
- Removed argument `index` from the `core.arrays.SparseArray` constructor (GH43523)
- Remove argument `squeeze` from `DataFrame.groupby()` and `Series.groupby()` (GH32380)
- Removed deprecated `apply`, `apply_index`, `__call__`, `onOffset`, and `isAnchored` attributes from `DateOffset` (GH34171)
- Removed `keep_tz` argument in `DatetimeIndex.to_series()` (GH29731)

- Changed behavior indexing on a `DataFrame` with a `DatetimeIndex` index using a string indexer, previously this operated as a slice on rows, now it operates like any other column key; use `frame.loc[key]` for the old behavior (GH36179)
- Enforced the `display.max_colwidth` option to not accept negative integers (GH31569)
- Removed the `display.column_space` option in favor of `df.to_string(col_space=...)` (GH47280)
- Removed the deprecated method `mad` from pandas classes (GH11787)
- Removed the deprecated method `tshift` from pandas classes (GH11631)
- Changed behavior of empty data passed into `Series`; the default dtype will be `object` instead of `float64` (GH29405)
- Changed the behavior of `DatetimeIndex.union()`, `DatetimeIndex.intersection()`, and `DatetimeIndex.symmetric_difference()` with mismatched timezones to convert to UTC instead of casting to object dtype (GH39328)
- Changed the behavior of `to_datetime()` with argument “now” with `utc=False` to match `Timestamp("now")` (GH18705)
- Changed the behavior of indexing on a timezone-aware `DatetimeIndex` with a timezone-naive `datetime` object or vice-versa; these now behave like any other non-comparable type by raising `KeyError` (GH36148)
- Changed the behavior of `Index.reindex()`, `Series.reindex()`, and `DataFrame.reindex()` with a `datetime64` dtype and a `datetime.date` object for `fill_value`; these are no longer considered equivalent to `datetime.datetime` objects so the reindex casts to object dtype (GH39767)
- Changed behavior of `SparseArray.astype()` when given a dtype that is not explicitly `SparseDtype`, cast to the exact requested dtype rather than silently using a `SparseDtype` instead (GH34457)
- Changed behavior of `Index.ravel()` to return a view on the original `Index` instead of a `np.ndarray` (GH36900)
- Changed behavior of `Series.to_frame()` and `Index.to_frame()` with explicit `name=None` to use `None` for the column name instead of the index’s name or default `0` (GH45523)
- Changed behavior of `concat()` with one array of `bool`-dtype and another of integer dtype, this now returns `object` dtype instead of integer dtype; explicitly cast the bool object to integer before concatenating to get the old behavior (GH45101)
- Changed behavior of `DataFrame` constructor given floating-point `data` and an integer `dtype`, when the data cannot be cast losslessly, the floating point dtype is retained, matching `Series` behavior (GH41170)
- Changed behavior of `Index` constructor when given a `np.ndarray` with object-dtype containing numeric entries; this now retains object dtype rather than inferring a numeric dtype, consistent with `Series` behavior (GH42870)
- Changed behavior of `Index.__and__()`, `Index.__or__()` and `Index.__xor__()` to behave as logical operations (matching `Series` behavior) instead of aliases for set operations (GH37374)
- Changed behavior of `DataFrame` constructor when passed a list whose first element is a `Categorical`, this now treats the elements as rows casting to `object` dtype, consistent with behavior for other types (GH38845)
- Changed behavior of `DataFrame` constructor when passed a `dtype` (other than int) that the data cannot be cast to; it now raises instead of silently ignoring the dtype (GH41733)
- Changed the behavior of `Series` constructor, it will no longer infer a datetime64 or timedelta64 dtype from string entries (GH41731)
- Changed behavior of `Timestamp` constructor with a `np.datetime64` object and a `tz` passed to interpret the input as a wall-time as opposed to a UTC time (GH42288)
- Changed behavior of `Timestamp.utcfromtimestamp()` to return a timezone-aware object satisfying `Timestamp.utcfromtimestamp(val).timestamp() == val` (GH45083)
- Changed behavior of `Index` constructor when passed a `SparseArray` or `SparseDtype` to retain that dtype instead of casting to `numpy.ndarray` (GH43930)
- Changed behavior of setitem-like operations (`__setitem__`, `fillna`, `where`, `mask`, `replace`, `insert`, fill_value for `shift`) on an object with `DatetimeTZDtype` when using a value with a non-matching timezone, the value will be cast to the object’s timezone instead of casting both to object-dtype (GH44243)
- Changed behavior of `Index`, `Series`, `DataFrame` constructors with floating-dtype data and a `DatetimeTZDtype`, the data are now interpreted as UTC-times instead of wall-times, consistent with how integer-dtype data are treated (GH45573)
- Changed behavior of `Series` and `DataFrame` constructors with integer dtype and floating-point data containing `NaN`, this now raises `IntCastingNaNError` (GH40110)
- Changed behavior of `Series` and `DataFrame` constructors with an integer `dtype` and values that are too large to losslessly cast to this dtype, this now raises `ValueError` (GH41734)

pandas.DataFrame.merge

DataFrame.merge(*right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=None, indicator=False, validate=None*) [source]

Merge DataFrame or named Series objects with a database-style join.

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

> **Warning**
>
> If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters:

right : *DataFrame or named Series*

Object to merge with.

how : *{'left', 'right', 'outer', 'inner', 'cross'}, default 'inner'*

Type of merge to be performed.

- left: use only keys from left frame, similar to a SQL left outer join; preserve key order.
- right: use only keys from right frame, similar to a SQL right outer join; preserve key order.
- outer: use union of keys from both frames, similar to a SQL full outer join; sort keys lexicographically.
- inner: use intersection of keys from both frames, similar to a SQL inner join; preserve the order of the left keys.
- cross: creates the cartesian product from both frames, preserves the order of the left keys.

> *New in version 1.2.0.*

on : *label or list*

Column or index level names to join on. These must be found in both DataFrames. If *on* is None and not merging on indexes then this defaults to the intersection of the columns in both DataFrames.

left_on : *label or list, or array-like*

Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

right_on : *label or list, or array-like*

Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

left_index : *bool, default False*

Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

right_index : *bool, default False*

Use the index from the right DataFrame as the join key. Same caveats as left_index.

sort : *bool, default False*

Sort the join keys lexicographically in the result DataFrame. If False, the order of the join keys depends on the join type (how keyword).

suffixes : *list-like, default is ("_x", "_y")*

A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in *left* and *right* respectively. Pass a value of *None* instead of a string to indicate that the column name from *left* or *right* should be left as-is, with no suffix. At least one of